Fpt University

TRƯỜNG ĐẠI HỌC FPT

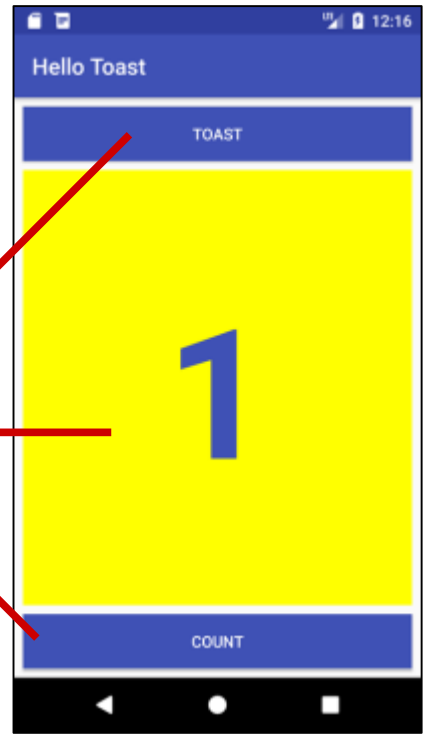
Simple UI Widgets

Contents

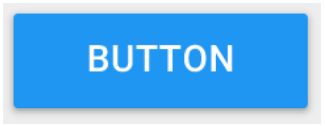
- Views
- ViewGroups
- View hierarchy

Views

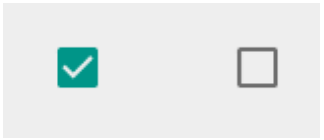
- **Everything you see is a view**
- **Examples of views**



Button



CheckBox



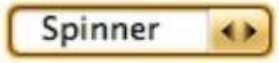
EditText

RadioButton

Slider

Switch

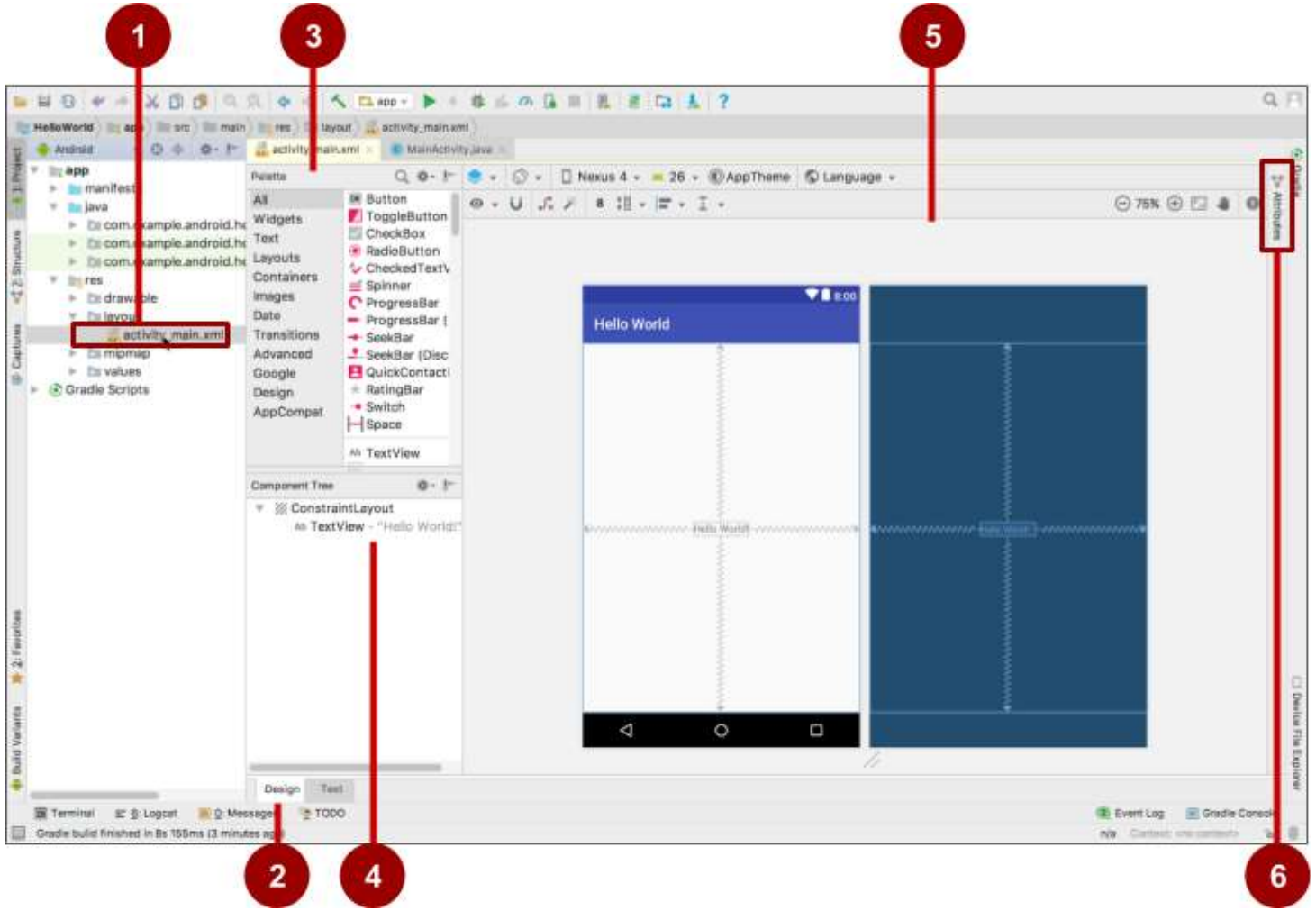
Android Widgets

 9:26:00 pm Analog/DigitalClock	 Button	 Checkbox	  Date/TimePicker
 EditText	 Gallery	 ImageView/Button	 ProgressBar
 RadioButton	 Spinner	 TextView	 MapView, WebView

Create views and layouts

- Android Studio layout editor: visual representation of XML
- XML editor
- Java code

Android Studio layout editor



- XML layout file
- Design and Text tabs
- Palette pane
- Component Tree
- Design and blueprint panes
- Attributes tab

View defined in XML

```
<TextView
        android:id="@+id/show_count"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/myBackgroundColor"
        android:text="@string/count_initial_value"
        android:textColor="@color/colorPrimary"
        android:textSize="@dimen/count_text_size"
        android:textStyle="bold"
/>
```

View attributes in XML

```
android:<property_name>="<property_value>"
Example: android:layout_width="match_parent"

android:<property_name>="@<resource_type>/resource_id"
Example: android:text="@string/button_label_next"

android:<property_name>="@+id/view_id"
Example: android:id="@+id/show_count"
```

Create View in Java code

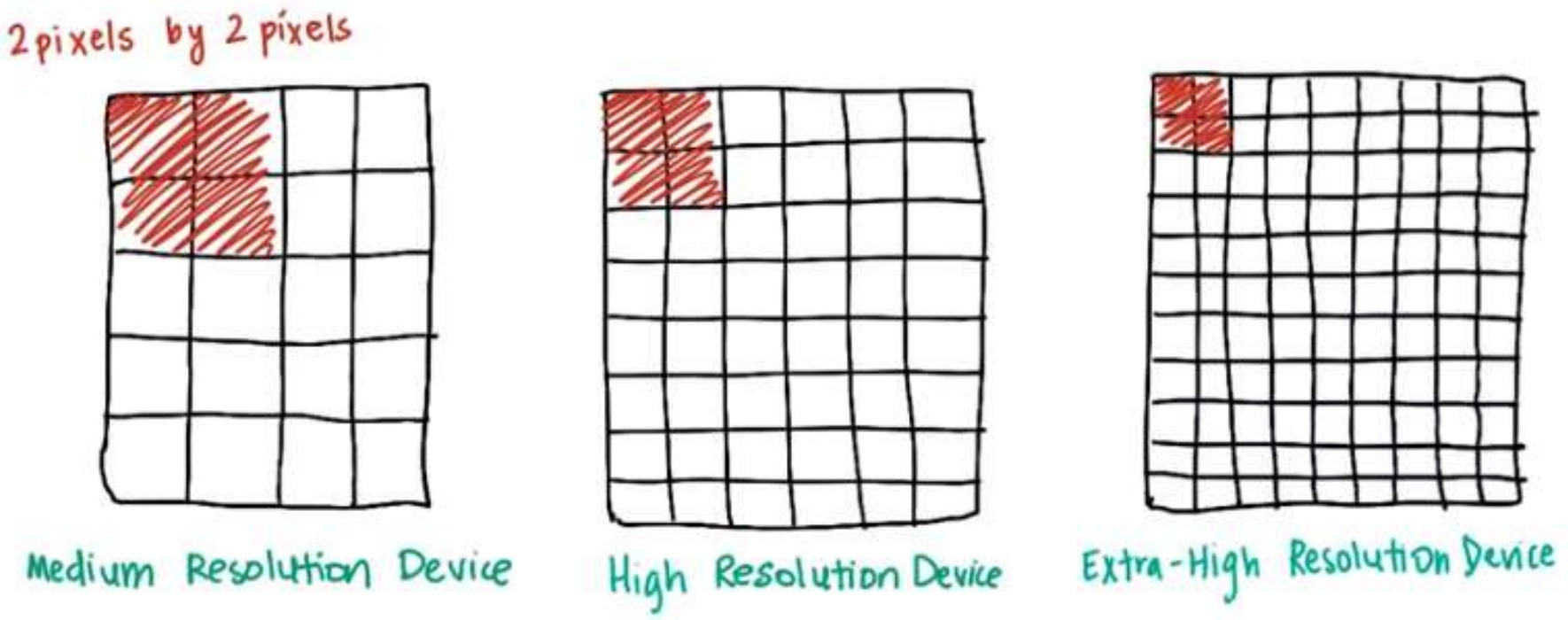
In an `Activity`:

```
TextView myText = new TextView(this);
myText.setText("Display this text!");
```

Views width


200 dp
wrap_content
match_parent
Happ
Birt
Happy
Birthday!
Happy
Birthday!

Density – independent pixels



Measure – inch and dp

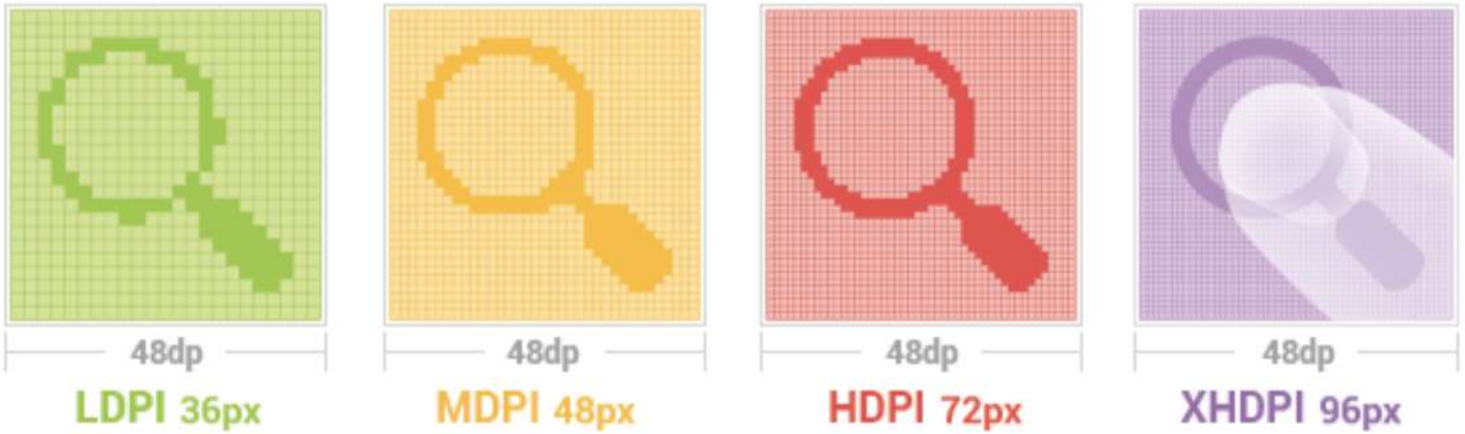
- inch = 2.54 cm
- dp or dip: (density-independent pixel)Pixels - lenght measure (similar with inch, Density-independent cm, mm)
- Base on 160dpi (Dots per inch), 1dp - 1px on a screen which has 160dpi.
- Dpi != 1, number of pixels in 1 dp = n * size/160.
- 160 dp = 1 inch –>

  1dp = 1/160 = 0.00625 inch

  100dp = 1.5785cm

  1cm = ? inch

Demonstrate px and dp

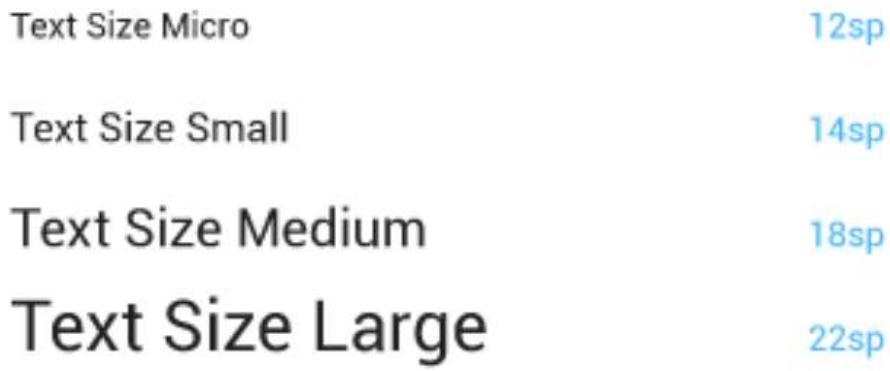


(Number of) px = (number of)dp * (dpi / 160)
Example dpi of screen is 320
10 dp has: 10 * (320/160) = 20 px, 1 dp has 2 px.

Measure - sp

- **sp: Scale-independent Pixels – Similar with dp, but scaled to suitable with size of fonts..**
- **The size of font is adjust base on 3 conditions:**
  - The adjust from programmer
  - screen density
  - Setting of font size on device.

Default: 1SP = 1DP.



Measure - dpi

- **Screen density : number of pixels on 1 inch of screen, called dpi (dots per inch)**
- **Base on dpi, the screen is classify:**
  - ldpi (low) ~120dpi
  - mdpi (medium) ~160dpi
  - hdpi (high) ~240dpi
  - xhdpi (extra-high) ~320dpi
  - xxhdpi (extra-extra-high) ~480dpi
  - xxxhdpi (extra-extra-extra-high) ~640dpi

Measurements

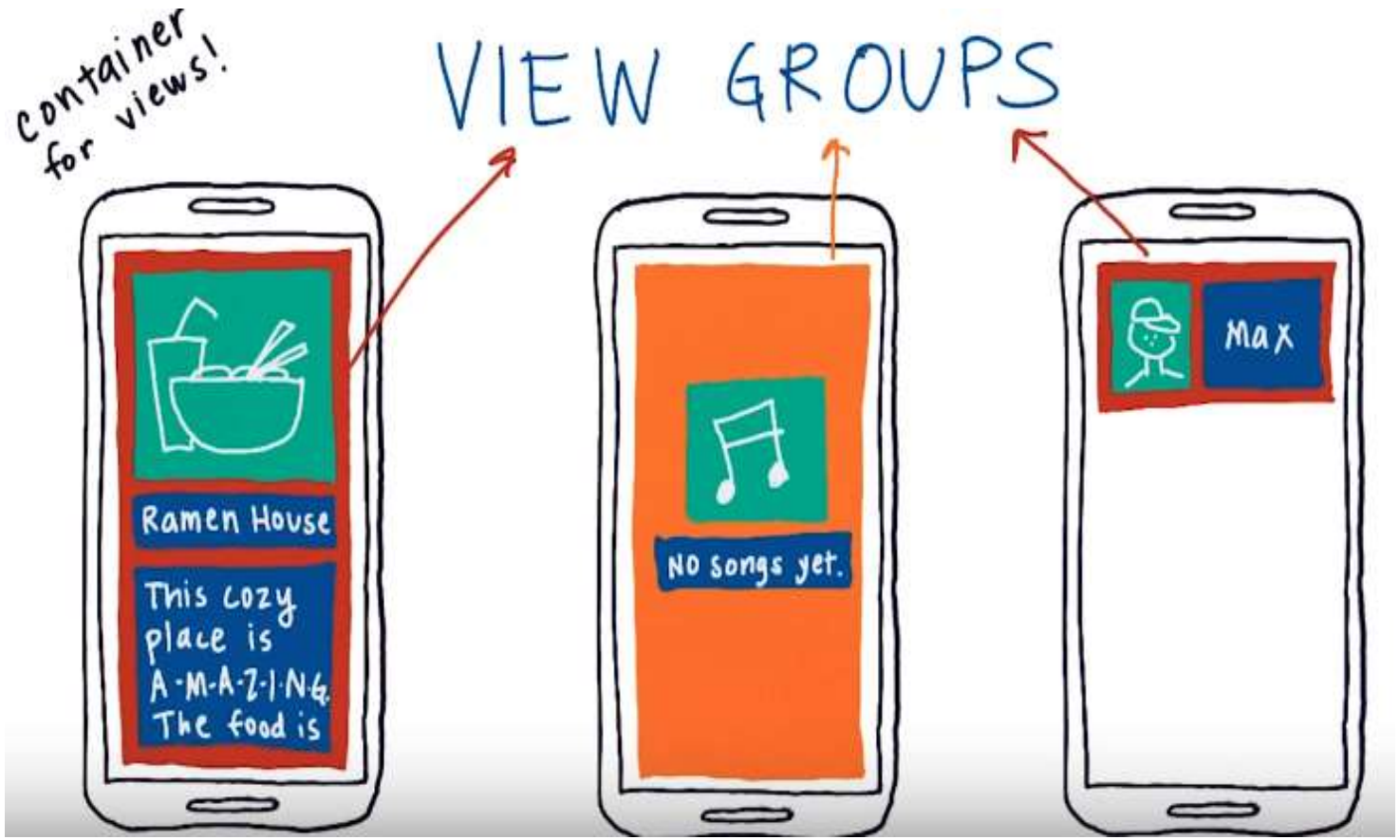
- **Density-independent Pixels (dp): for Views**
- **Scale-independent Pixels (sp): for text**
- **Don't use device-dependent or density-dependent units:**
  - Actual Pixels (px)
  - Actual Measurement (in, mm)
  - Points - typography 1/72 inch (pt)

Common Widgets

- Button
- TextView
- EditText
- ImageView
- CheckBox
- RadioButton
- WebView
- AdapterView
  - ListView
  - Spinner
- RecyclerView
- …

ViewGroups

- **ViewGroup contains "child" views**



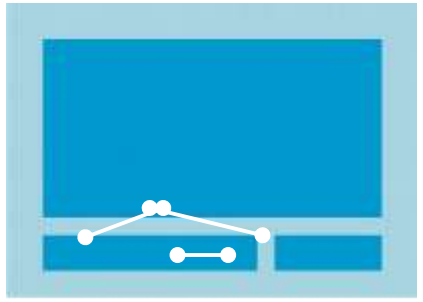
ViewGroups

- **Common Layout Classes**
  - `ConstraintLayout`: Connect views with constraints
  - `LinearLayout`: Horizontal or vertical row
  - `TableLayout`: Rows and columns
  - `FrameLayout`: Shows one child of a stack of children

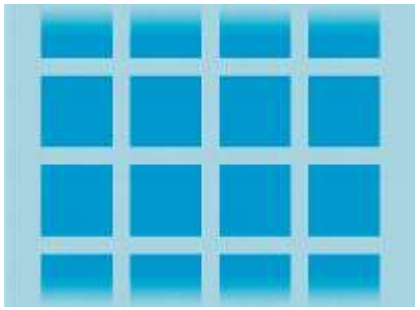
ViewGroups

- **Common Layout Classes**

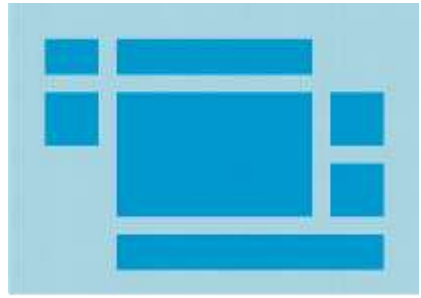
`LinearLayout`



`ConstraintLayout`

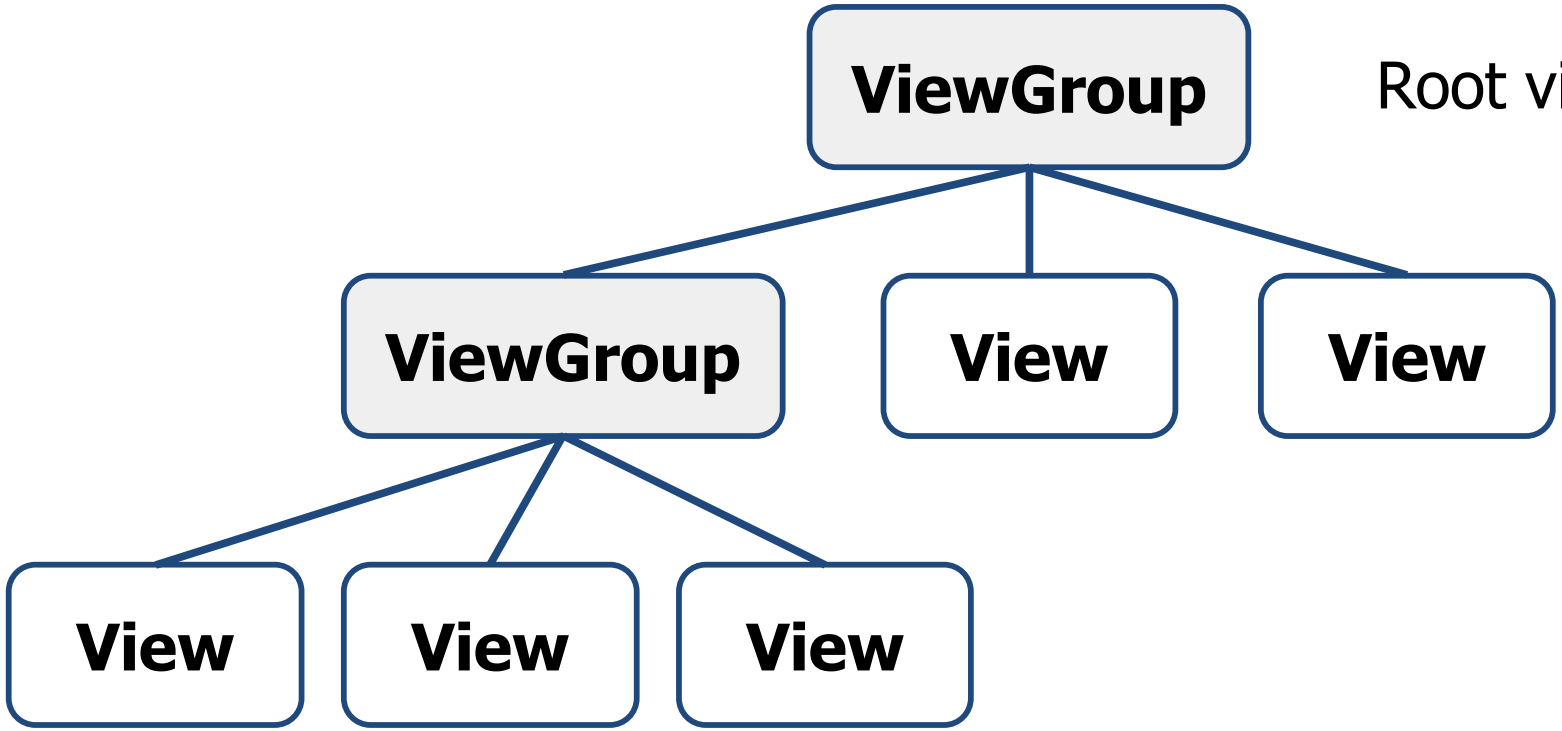


`GridLayout`



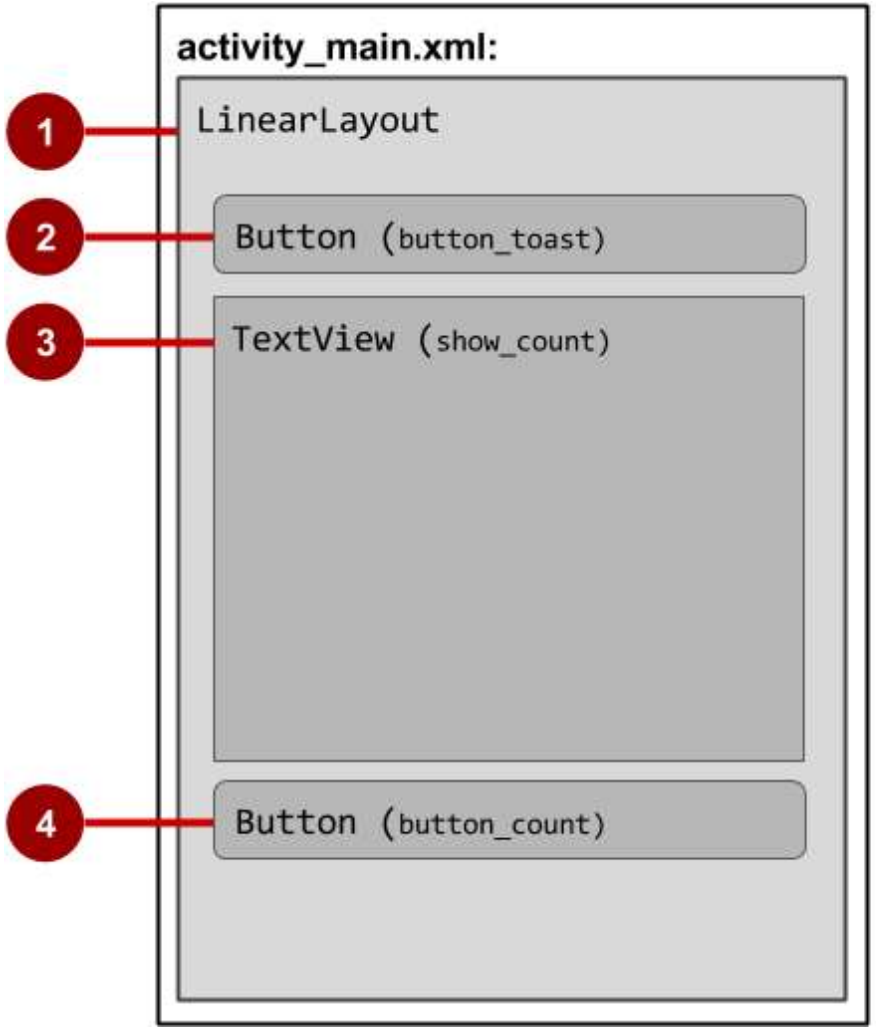
`TableLayout`

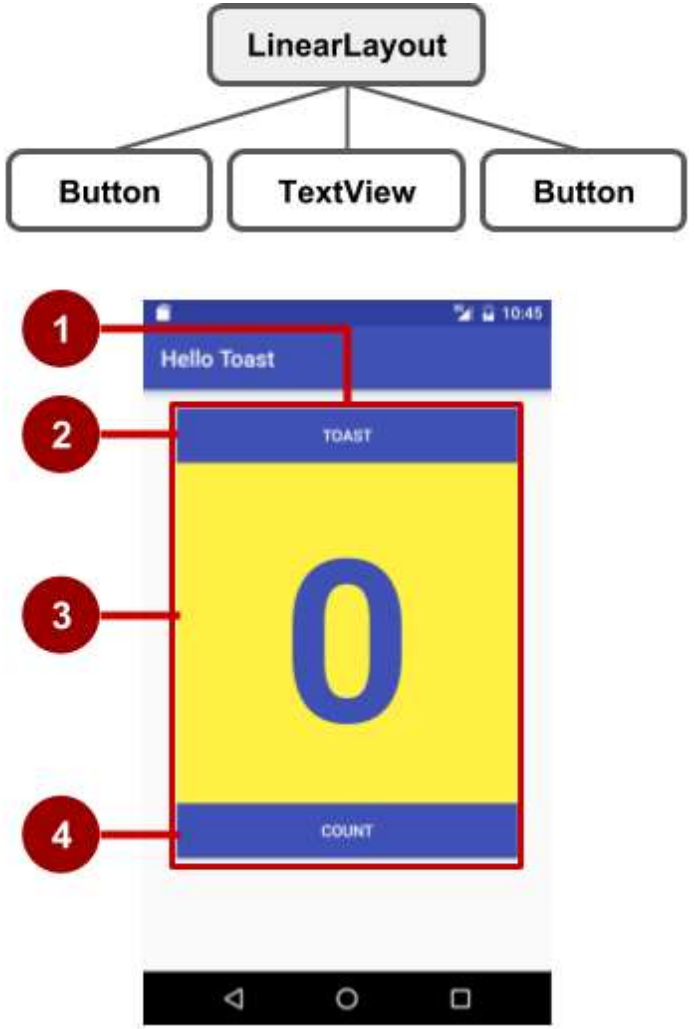
Hierarchy of viewgroups and views



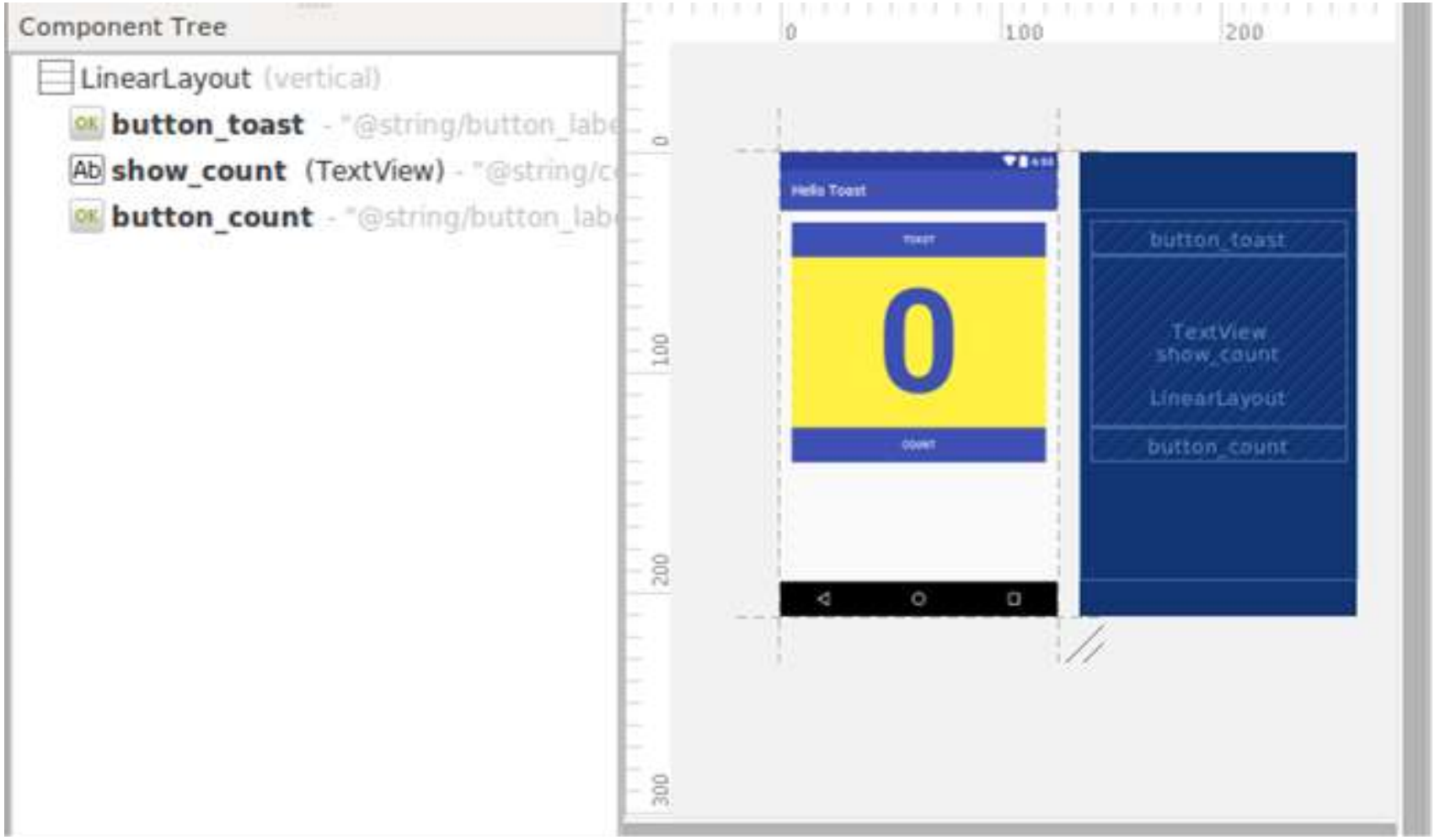
Root view is always a `ViewGroup`

View hierarchy and screen layout





View hierarchy in the layout editor



Layout created in XML

```
<LinearLayout
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent">
    <Button
        ... />
    <TextView
        ... />
    <Button
        ... />
</LinearLayout
```

Layout created in Java Activity code

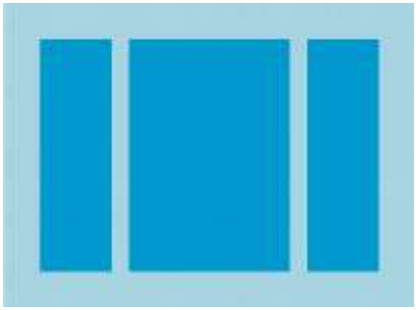
```
LinearLayout linearL = new LinearLayout(this);
linearL.setOrientation(LinearLayout.VERTICAL);

TextView myText = new TextView(this);
myText.setText("Display this text!");

linearL.addView(myText);
setContentView(linearL);
```

Set width and height in Java code

```
LinearLayout.LayoutParams layoutParams =
  new Linear.LayoutParams(
      LayoutParams.MATCH_PARENT,
      LayoutParams.MATCH_CONTENT);
myView.setLayoutParams(layoutParams);
```

LinearLayout

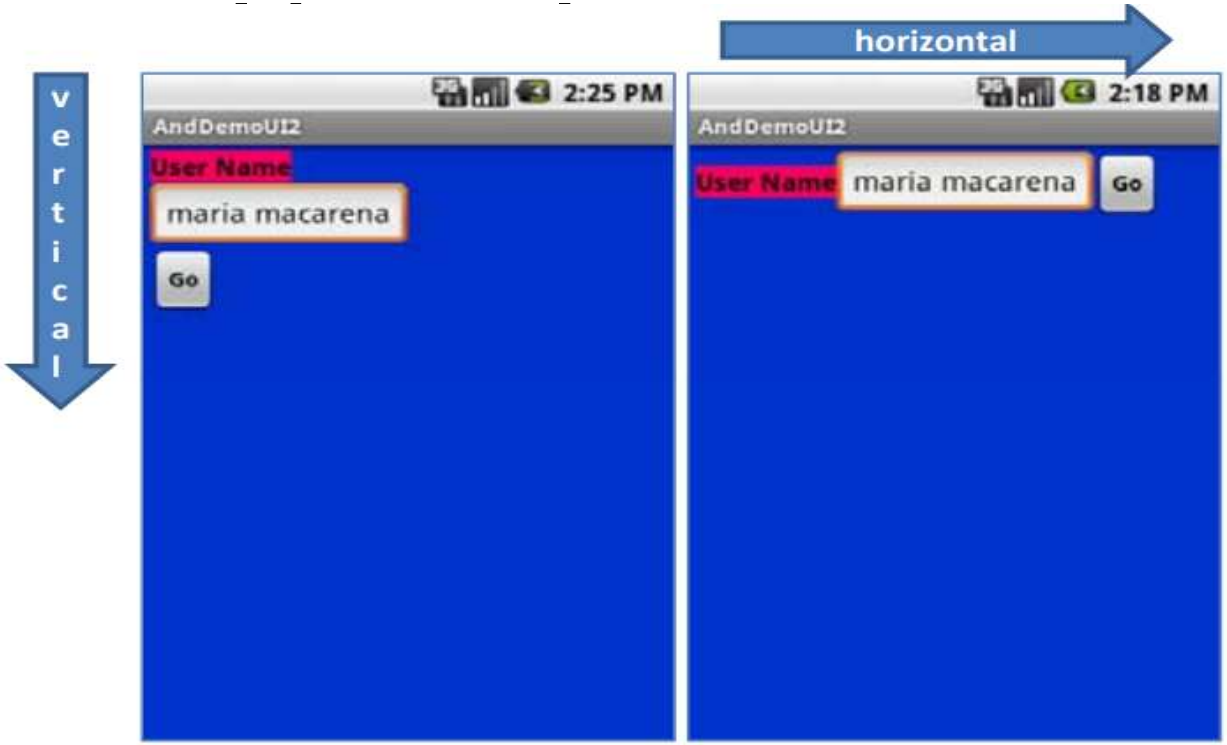
- **A box model – widgets or child containers are lined up in a column or row, one after the next.**
- **Special attributes:**
  - orientation: indicates whether the LinearLayout represents a row or a column (vertical or horizontal)
  - weight: indicates whether the LinearLayout represents a row or a column
  - gravity: is used to indicate how a control will align on the screen



LinearLayout

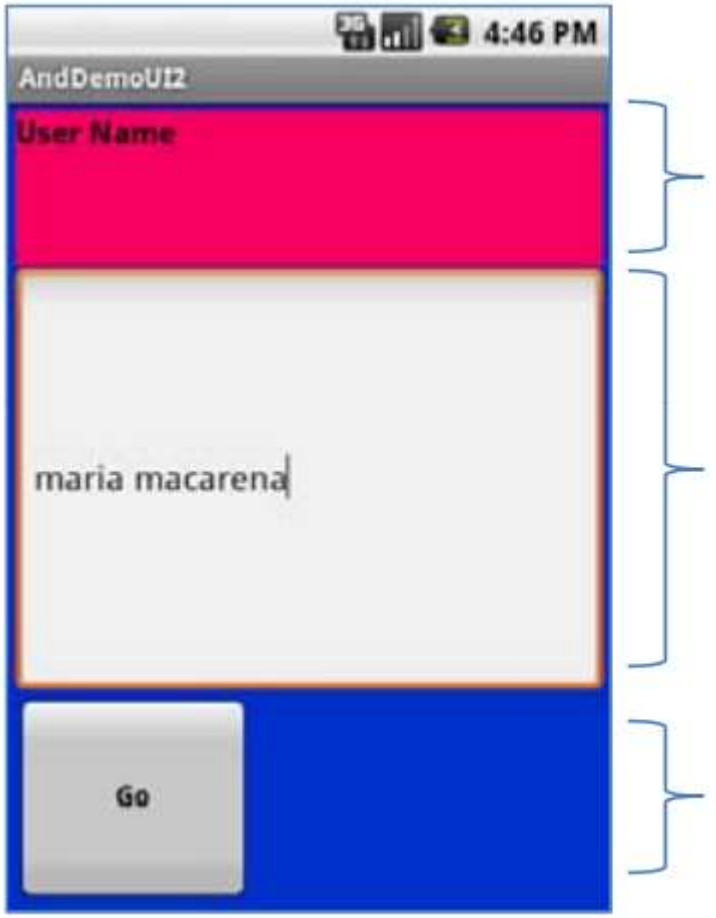
LinearLayout - Special attributes

- **orientation**
  - Name: android:orientation
  - Value



LinearLayout - Special attributes

- **weight**
  - Name: android:layout_weight
  - Value: 1, 2, 3, …



LinearLayout - Special attributes

- **gravity**
  - Name: android:layout_gravity
  - Value: left, center, right, top, bottom, (

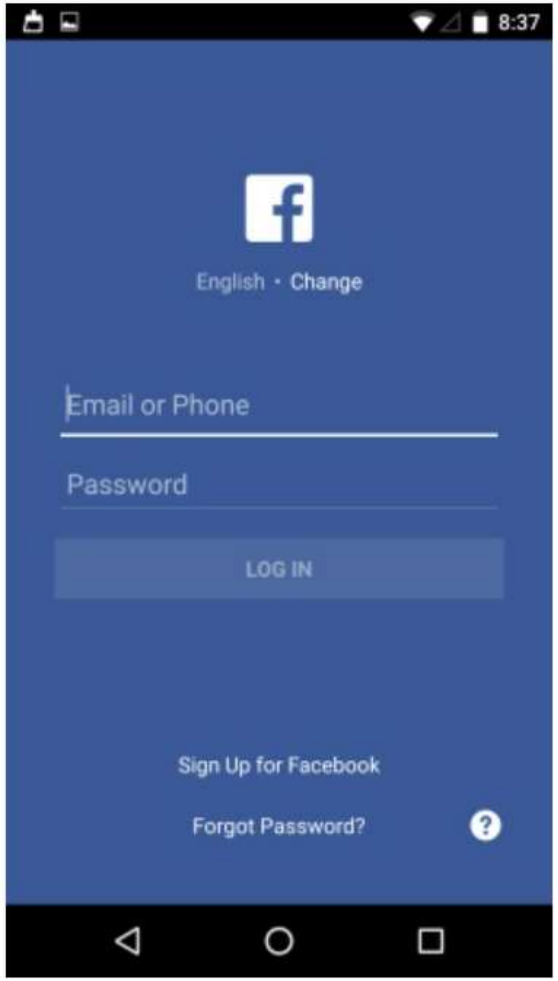
ConstraintLayout

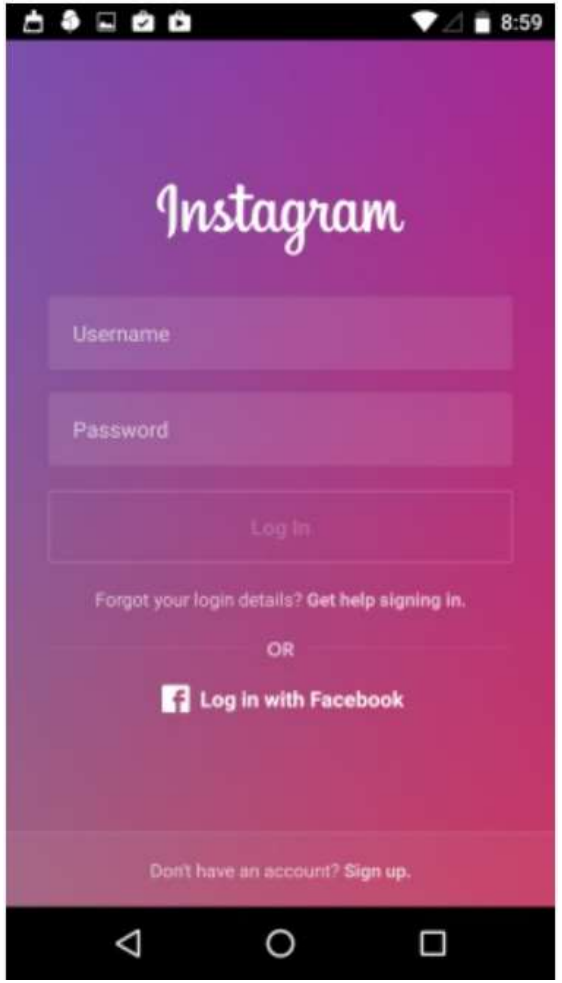
Self-study

- https://developer.android.com/training/constraint-layout
- https://developer.android.com/codelabs/constraint-layout#0

Classwork

- **Design one of Login Screens as below:**





References

- https://pttrns.com/android-patterns
- https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/